



**UNIVERSIDADE
FEDERAL DO CARIRI**

Viewing 2D

Professora Dra. Luana Batista da Cruz
luana.batista@ufca.edu.br

Roteiro

01 Introdução

02 A janela de recorte

03 Algoritmos de recorte

04 Programação OpenGL

01

Introdução

Viewing pipeline 2D

Janela de recorte

Viewport

Transformação 2D da visão

Introdução

- **Viewing pipeline 2D**

- Processo para criar a visão 2D de uma cena, determinando quais partes serão mostradas e suas localizações na tela

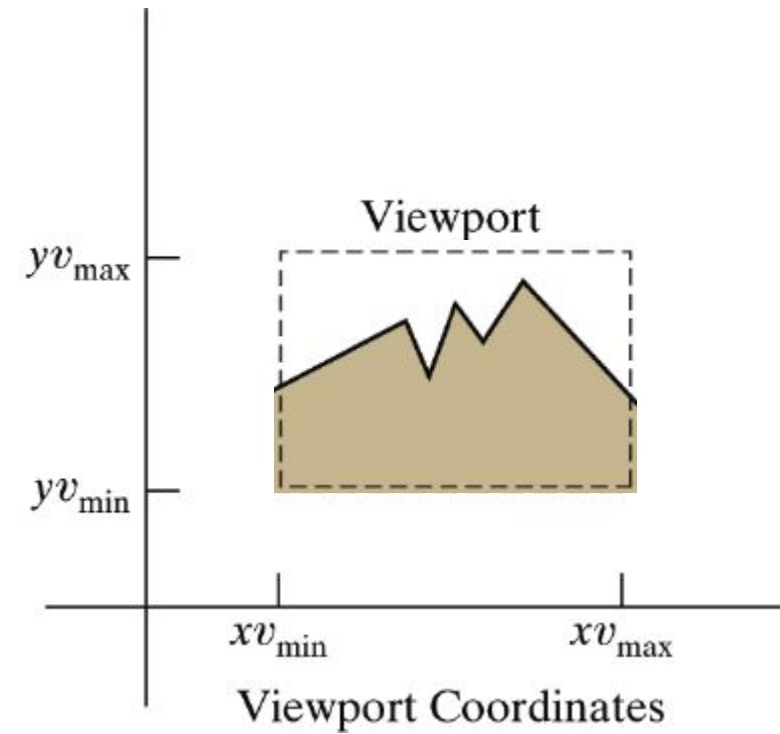
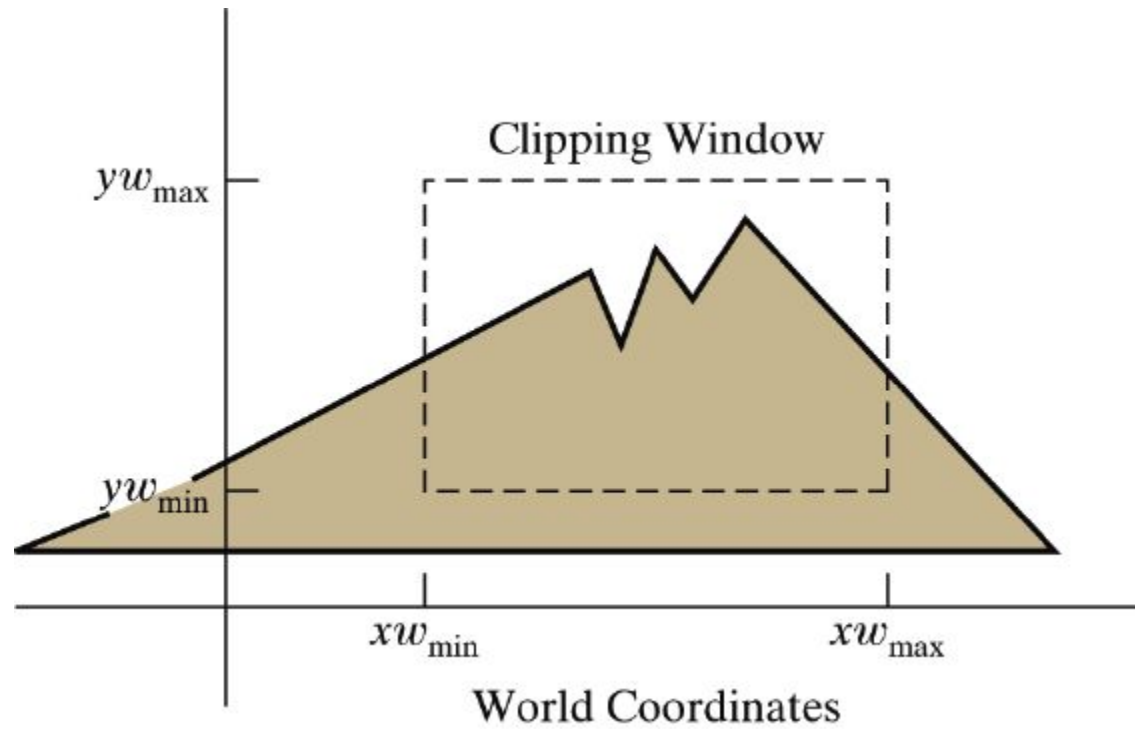
- A imagem é determinada no **sistema de coordenadas do mundo (world coordinates)** cujas partes especificadas (selecionadas e transcritas) são mapeadas para o **sistema de coordenadas do dispositivo (device coordinates)**

- Esse mapeamento envolve uma série de translações, rotações e escalas
- Assim como operações para eliminar as partes da imagem que estão fora da área de visão

Introdução

- **Janela de recorte ou clipping window**
 - Uma seção de uma cena 2D que é selecionada para ser mostrada
 - Tudo o que estiver fora dessa seção será “cortado fora”
- **Viewport**
 - A janela de recorte é posicionada dentro de uma janela do sistema usando outra janela chamada de **Viewport**
 - Objetos dentro da janela de recorte (o que será visto) são mapeados para a **Viewport**, que por sua vez é posicionada dentro da janela do sistema (onde serão vistos)
 - **Múltiplas Viewports** podem ser usadas para mostrar diferentes seções da imagem em diferentes posições

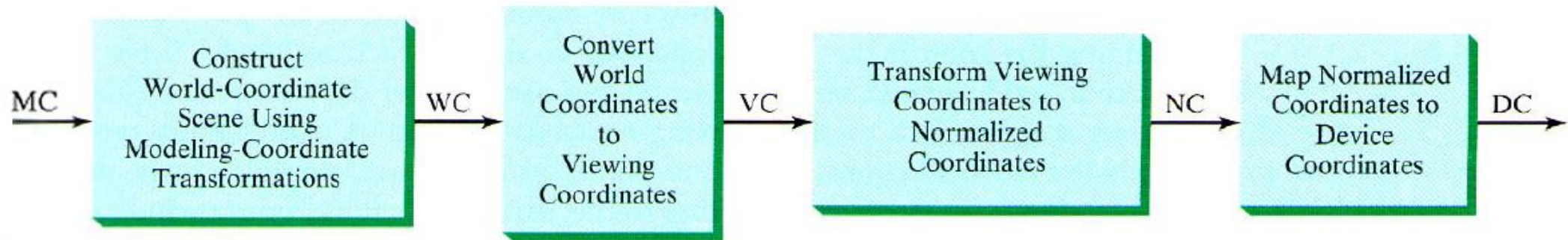
Introdução



Introdução

- **Transformação 2D da visão**

- Mapeamento de uma descrição da cena no sistema de coordenadas do mundo para o sistema de coordenadas do dispositivo



- Para tornar o processo independente dos requisitos de diferentes dispositivos de saída, sistemas gráficos convertem a descrição dos objetos para coordenadas normalizadas (entre 0 e 1 ou entre -1 e 1)

02

A janela de recorte

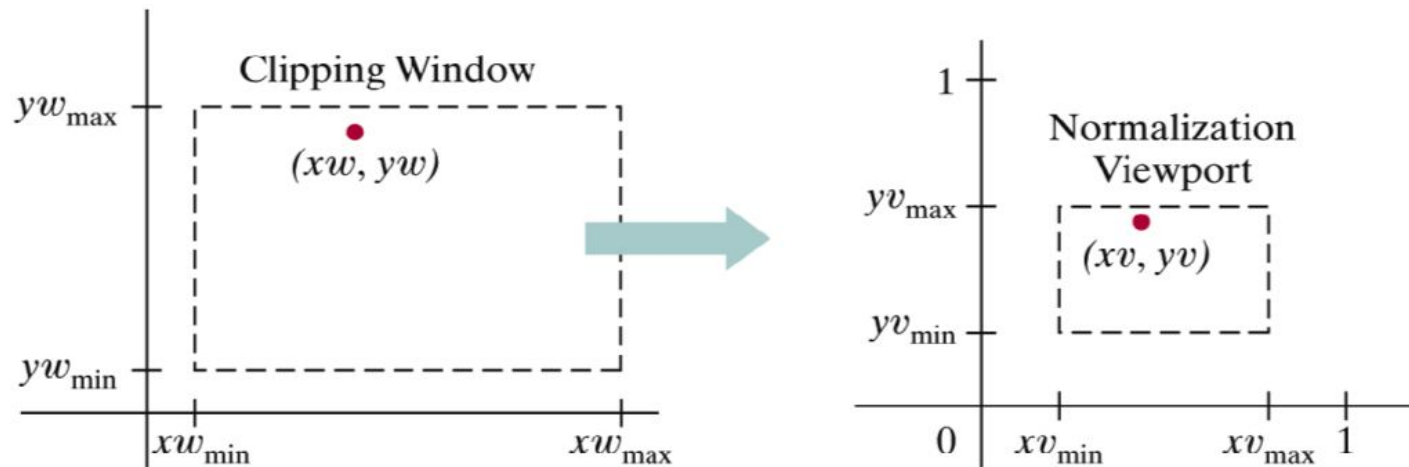
Mapeando a janela de recorte em uma viewport normalizada

A janela de recorte

- Embora seja possível criar **janelas de recorte** de qualquer formato, a maioria das APIs gráficas somente suporta janelas retangulares alinhadas aos eixos x e y devido o custo computacional
- No OpenGL, a **janela de recorte** é especificada no sistema de coordenadas do mundo

Mapeando a janela de recorte em uma Viewport normalizada

- Considerando uma **viewport** com as coordenadas entre 0 e 1, temos que mapear a descrição dos objetos para esse espaço normalizado usando transformações que mantenham a posição relativa de um ponto como foi definida na **janela de recorte**



- O ponto (xw, yw) é mapeado para (xv, yv)

Mapeando a janela de recorte em uma Viewport normalizada

- Para transformar um ponto no sistema de coordenadas do mundo para um ponto na viewport normalizada, temos que fazer

$$\frac{x_v - x_{v_{min}}}{x_{v_{max}} - x_{v_{min}}} = \frac{x_w - x_{w_{min}}}{x_{w_{max}} - x_{w_{min}}}$$

$$\frac{y_v - y_{v_{min}}}{y_{v_{max}} - y_{v_{min}}} = \frac{y_w - y_{w_{min}}}{y_{w_{max}} - y_{w_{min}}}$$

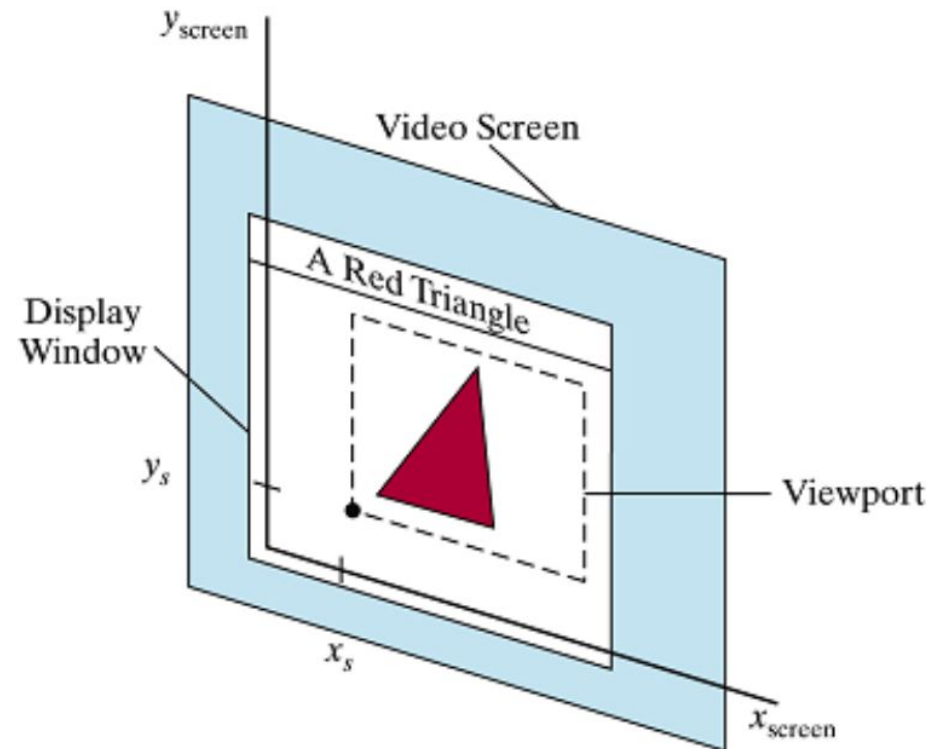
A primeira equação é para encontrar a coordenada **x_v**, e a segunda para encontrar a coordenada **y_v**

Mapeando a janela de recorte em uma Viewport normalizada

- Neste mapeamento, as **posições relativas** dos objetos são mantidas
 - Um objeto dentro da janela de recorte estará dentro da viewport
 - Matematicamente, esse processo se refere à translação e à escala
- As proporções relativas dos objetos só serão mantidas se a razão de aspecto da viewport for igual a da janela de recorte

Mapeando a janela de recorte em uma Viewport normalizada

- O último passo consiste em posicionar a área da viewport na janela da tela



Algoritmos de recorte

Recorte de ponto 2D

Recorte de linha 2D

Recorte de linha de Cohen-Sutherland

Recorte de polígonos 2D

Recorte de polígonos de Sutherland-Hodgman

Algoritmos de recorte

- No **viewing pipeline** serve para extrair uma porção designada de uma cena para ser apresentada em um dispositivo de saída
- Identifica as partes de uma imagem que estão fora da janela de recorte, eliminando essas da descrição da cena que é passada para o dispositivo de saída. Portanto, se tiver objetos fora, não é necessário realizar cálculos para passar para a viewport
- O recorte é aplicado sobre a janela de recorte

Algoritmos de recorte

- Existem diversos algoritmos para o recorte de
 - Pontos
 - Linhas (segmentos de reta)
 - Áreas-preenchidas (polígonos)
 - Curvas
 - Texto
- Os três primeiros são componentes padrão dos pacotes gráficos
 - Maior rapidez de processamento se as fronteiras dos objetos forem segmentos de reta

Algoritmos de recorte

- A região de recorte será uma janela retangular na posição padrão, com arestas de fronteira em xw_{min} , xw_{max} , yw_{min} e yw_{max}
 - Caso não seja configurado corresponderá ao quadrado normalizado entre 0 e 1 ou -1 e 1

Recorte de ponto 2D

- Dado um ponto $P(x, y)$ esse será apresentado no dispositivo de saída se e somente se

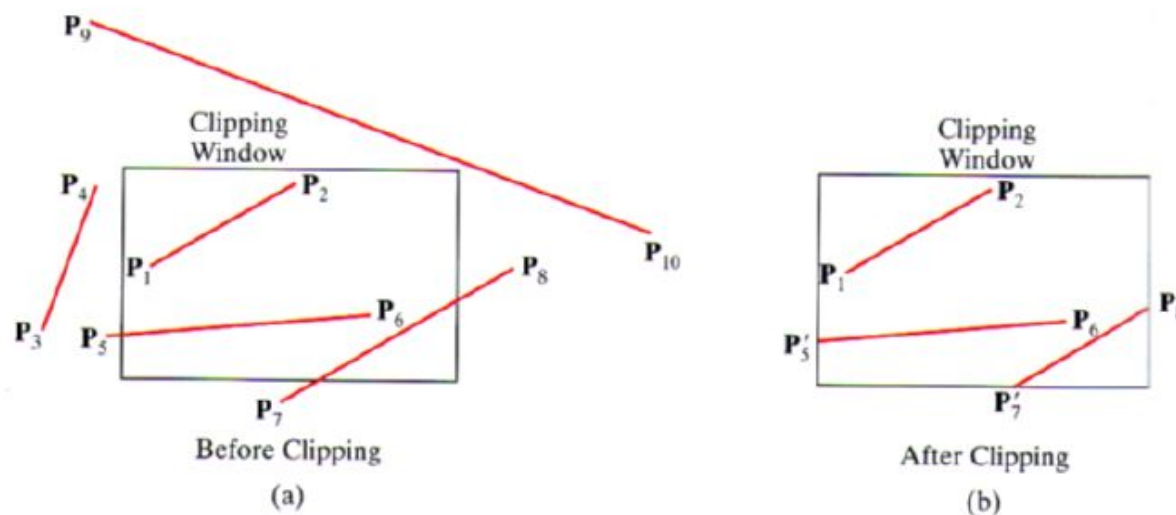
$$xW_{min} \leq x \leq xW_{max}$$

$$yW_{min} \leq y \leq yW_{max}$$

- Esse processo é especialmente útil para cortes em sistemas de partículas, como nuvens, fumaças, explosões, etc

Recorte de linha 2D

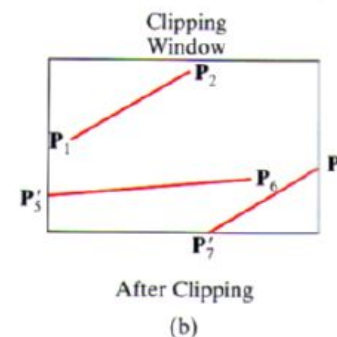
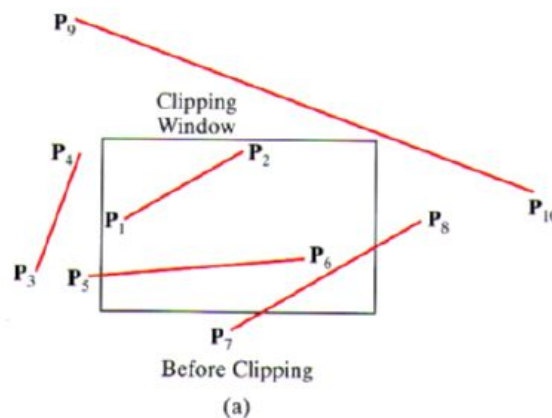
- Processa cada linha em uma cena por meio de uma série de testes e **cálculos de intersecção** para determinar se uma linha ou parte dela precisa ser desenhada



- A tarefa mais cara computacionalmente do recorte é calcular as intersecções das linhas com a janela de recorte
 - Portanto, o objetivo é **minimizar o cálculo de intersecções**

Recorte de linha 2D

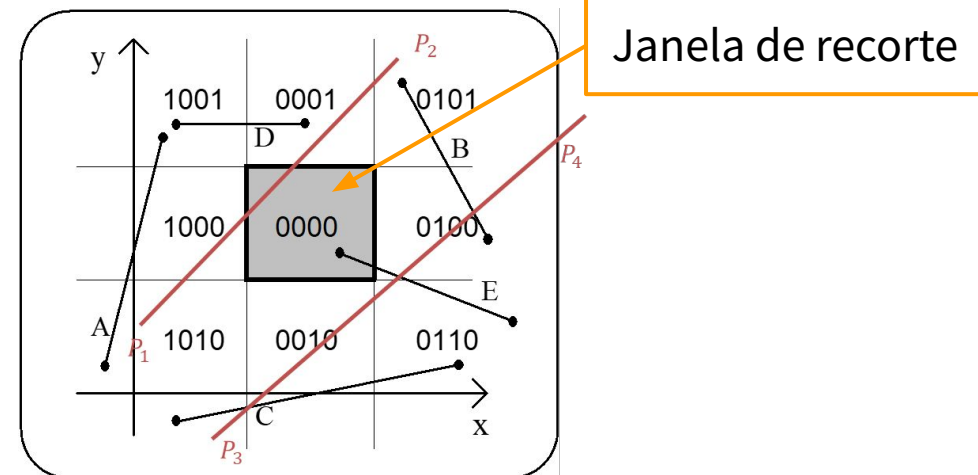
- Análises básicas
 - Quando os dois pontos limitantes de uma linha estão dentro da janela (linha P_1P_2), a linha está **completamente dentro**
 - Quando os dois pontos limitantes estão fora de uma das quatro fronteiras (linha P_3P_4), a linha está **completamente fora**
 - Se ambos testes falham, o segmento de linha pode ou não cruzar o interior da janela



Recorte de linha de Cohen-Sutherland

- Um dos primeiros algoritmos para acelerar o processo de recorte
- O tempo de recorte é reduzido executando mais testes antes dos cálculos das intersecções
- Inicialmente a cada ponto final das linhas (vértices) é codificado com um valor binário de 4 dígitos (bits)

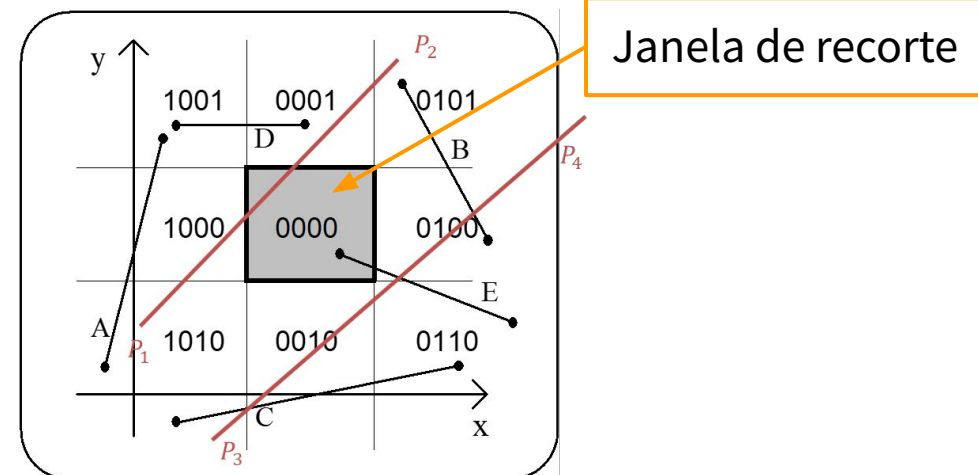
E D A b A c



Recorte de linha de Cohen-Sutherland

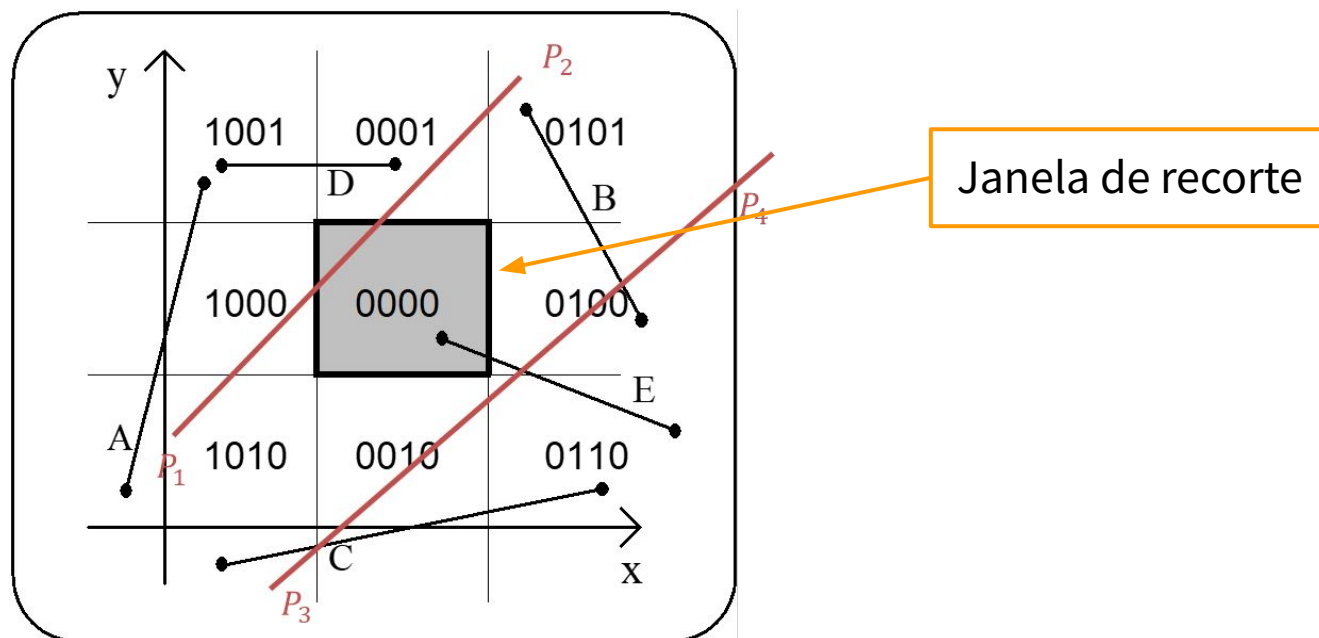
- Os valores dos bits são determinados comparando as coordenadas (x, y) do ponto com as fronteiras de recorte
 - O bit da **esquerda** é definido como 1 se $x < xw_{\min}$ (fronteira da esquerda)
 - O bit da **direita** é definido como 1 se $x > xw_{\max}$ (fronteira direita)
 - O bit **abaixo** é definido como 1 se $y < yw_{\min}$ (fronteira abaixo)
 - O bit **acima** é definido como 1 se $y > yw_{\max}$ (fronteira acima)

E D Ab Ac



Recorte de linha de Cohen-Sutherland

- As 4 fronteiras juntas criam **nove regiões de separação** do sistema de coordenadas

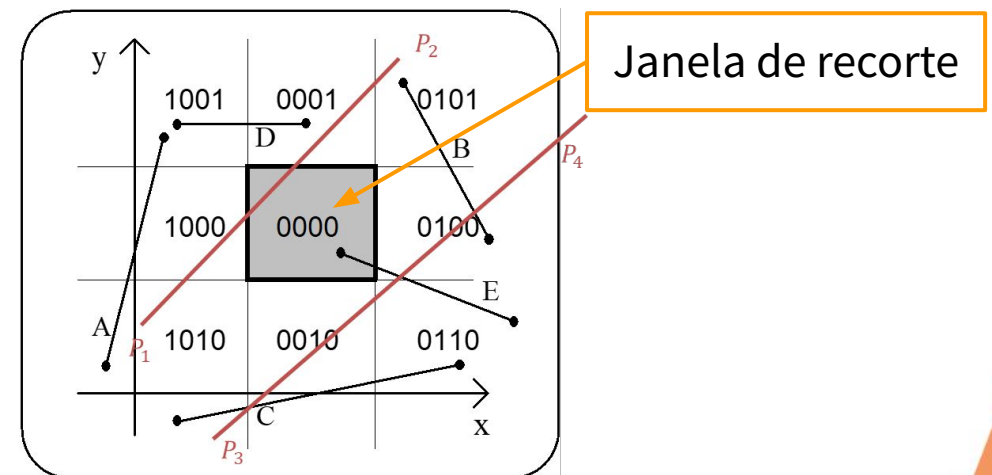


- Ex: um ponto abaixo e à esquerda da janela de recorte recebe valor 1010, um ponto dentro 0000

Recorte de linha de Cohen-Sutherland

- Com base nesses códigos é possível determinar rapidamente se uma linha está completamente fora ou dentro da janela de recorte (**casos triviais**)
 - Linhas **completamente dentro** têm seus pontos definidos como 0000
 - Linhas **completamente fora** da janela têm seus pontos definidos com valor 1 bit nas mesmas posições dos quatro códigos (E, D, Ab, Ac)
 - Uma linha com pontos finais identificados por 1001 e 0001 (linha D) está completamente acima da janela de recorte (operação AND)

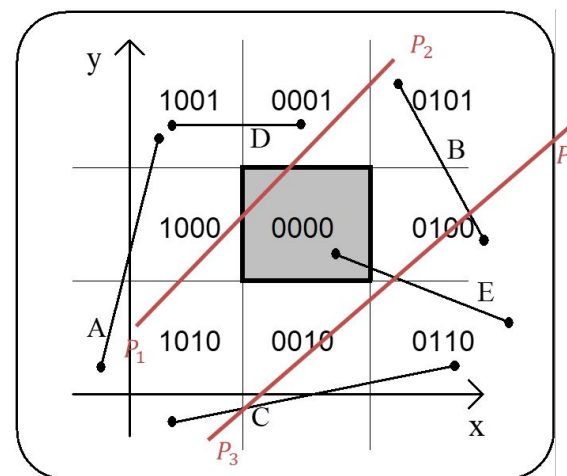
E D Ab Ac



Recorte de linha de Cohen-Sutherland

- As linhas que não podem ser identificadas como completamente fora ou dentro da janela de recorte são processadas para verificar intersecções e recortar (**casos não triviais**)
 - Quando faz a operação AND e o resultado é 0000 (indefinido)

$\overline{E} \overline{D} \overline{A} \overline{C}$



Recorte de linha de Cohen-Sutherland

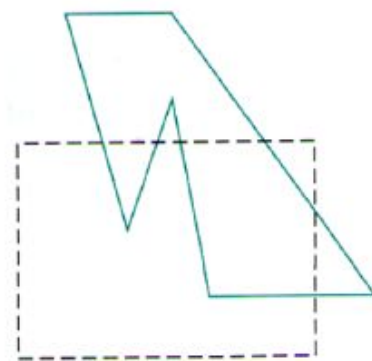
- As linhas que não podem ser identificadas como completamente fora ou dentro da janela de recorte são processadas para verificar intersecções e recortar (**casos não triviais**)
 - Move vértice inicial da linha para a próxima fronteira da janela de recorte por meio do **cálculo da intersecção** (abscissa ou ordenada) e **codifica novamente**
 - **Parcialmente dentro:** se o novo vértice recebe o código 0000, a linha é cortada nesse ponto, e o algoritmo prossegue verificando o segundo vértice até que ambos estejam dentro da janela de recorte (recortados)
 - **Completamente fora:** se a operação lógica AND entre os códigos do novo vértice e do outro vértice resultar em uma posição bit 1, o segmento é descartado
 - Repete até que o segmento seja completamente recortado ou descartado (ou seja, até cair em um dos casos triviais)

Recorte de linha de Cohen-Sutherland

- Conforme cada intersecção com as fronteiras da janela de recorte são calculadas, a linha é recortada até restar apenas o que está dentro da janela ou descartar o segmento de reta

Recorte de polígonos 2D

- Para fazer o corte de polígonos, os algoritmos de recorte de linhas não podem ser aplicados porque em geral não produziriam polígonos fechados
 - Produziriam linhas desconexas sem informação de como uni-las para formar o polígono recortado



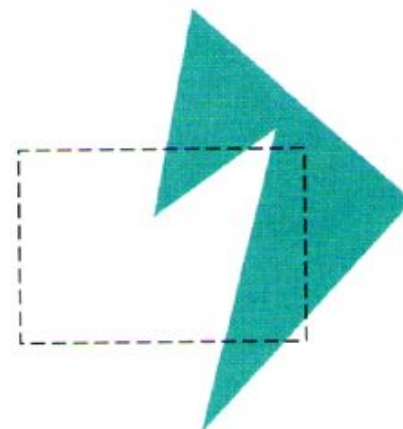
Before Clipping

(a)



After Clipping

(b)



Before Clipping

(a)

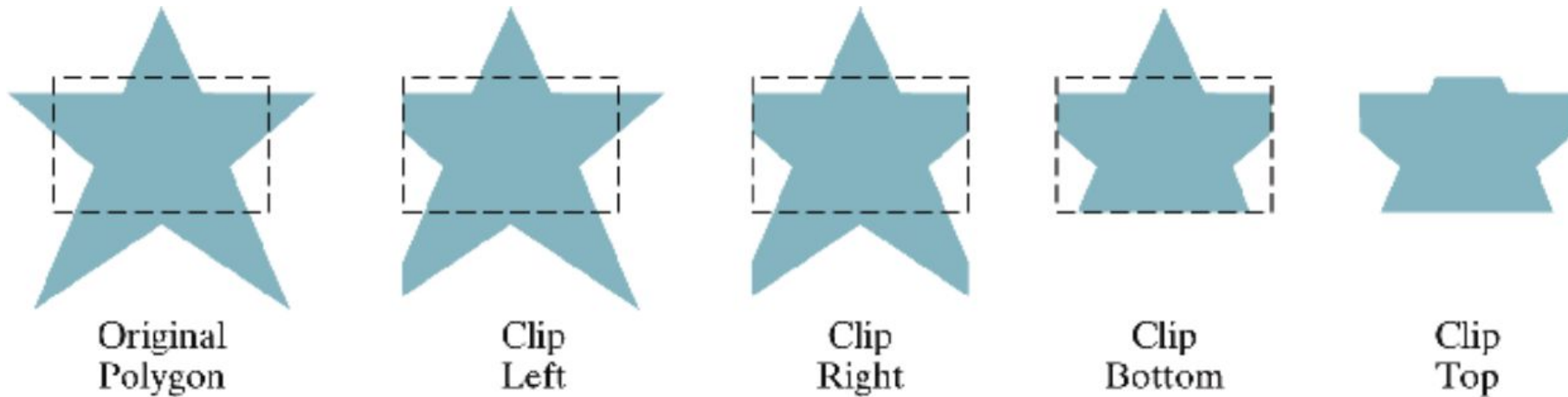


After Clipping

(b)

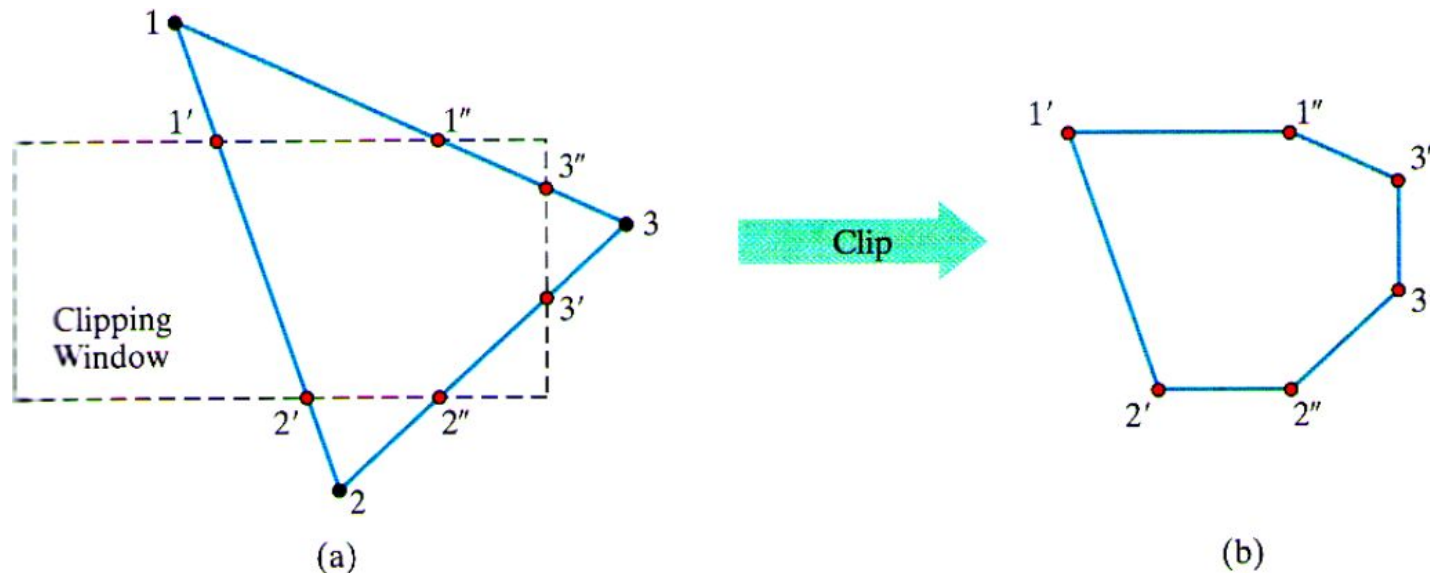
Recorte de polígonos 2D

- É possível processar o polígono contra as fronteiras da janela de recorte (E, D, Ab, Ac) de forma semelhante ao algoritmo de recorte de linhas
 - Isso é feito determinando o novo formato do polígono cada vez que uma fronteira de recorte é processada



Recorte de polígonos 2D

- Uma forma simples de realizar o recorte de **polígonos convexos** é criar uma nova lista de vértices a cada recorte realizado contra uma fronteira, e então passar essa lista para o próximo recorte, contra outra fronteira
- Para **polígonos côncavos** o processo é mais complexo podendo resultar em múltiplas listas de vértices



Recorte de polígonos de Sutherland-Hodgman

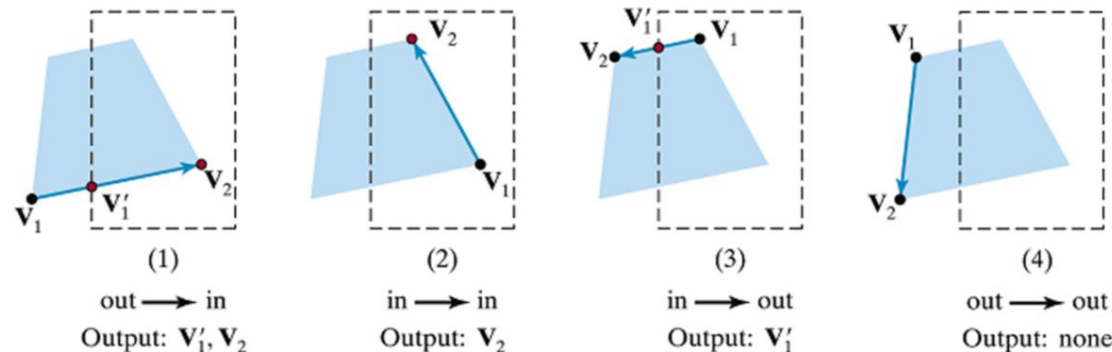
- Uma forma eficiente de realizar esse recorte é mandar os vértices dos polígonos para cada estágio de recorte de forma que os vértices recortados possa ser passado imediatamente para o próximo estágio
- A **estratégia** deste algoritmo é mandar os **pares de pontos finais de cada linha** sucessiva do polígono para uma **série de recortadores** (esquerda, direita, inferior e superior)
 - Conforme o recorte é executado para um par de vértices, as coordenadas recortadas são enviadas para o próximo recortador

Recorte de polígonos de Sutherland-Hodgman

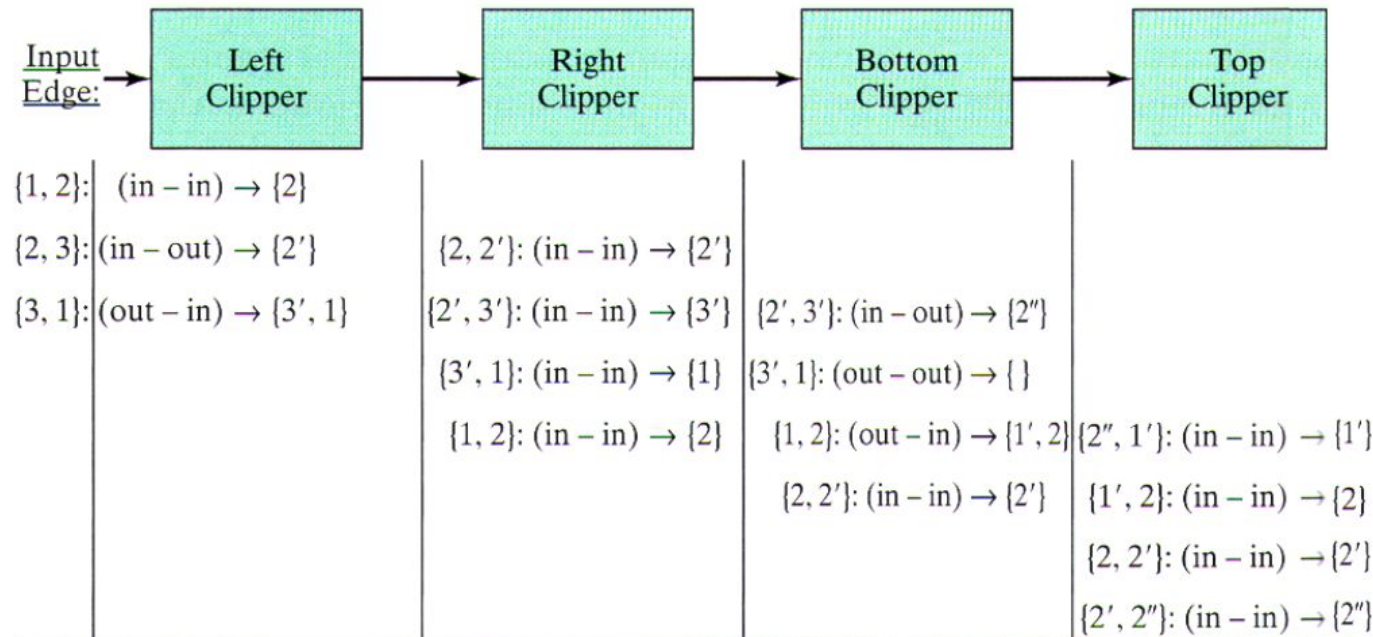
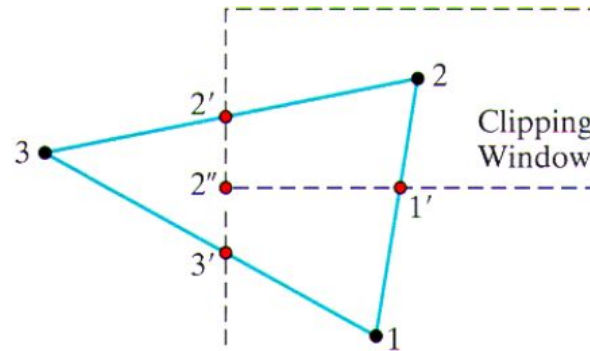
- As arestas dos polígonos precisam ser processadas uma a uma em ordem **anti-horária** para cada uma das fronteiras
- Existem 4 diferentes casos que precisam ser considerados quando uma aresta do polígono é processada
 - 1) O primeiro ponto da aresta está fora da janela de recorte e o segundo dentro
 - 2) Ambos pontos estão dentro da janela de recorte
 - 3) O primeiro ponto da aresta está dentro da janela de recorte e o segundo fora
 - 4) Ambos pontos estão fora da janela de recorte

Recorte de polígonos de Sutherland-Hodgman

- Conforme cada par de vértices sucessivos é passado para um dos recortadores, a saída é gerada para o próximo recortador de acordo com os seguintes testes
 - Se o primeiro vértice está fora da janela e o segundo dentro, é mandado para o próximo recortador a intersecção obtida e o segundo vértice
 - Se ambos vértices estão dentro, somente o segundo vértice é enviado
 - Se o primeiro vértice está dentro da janela e o segundo fora, é mandado para o próximo recortador somente a intersecção
 - Se ambos vértices estão fora, nada é enviado

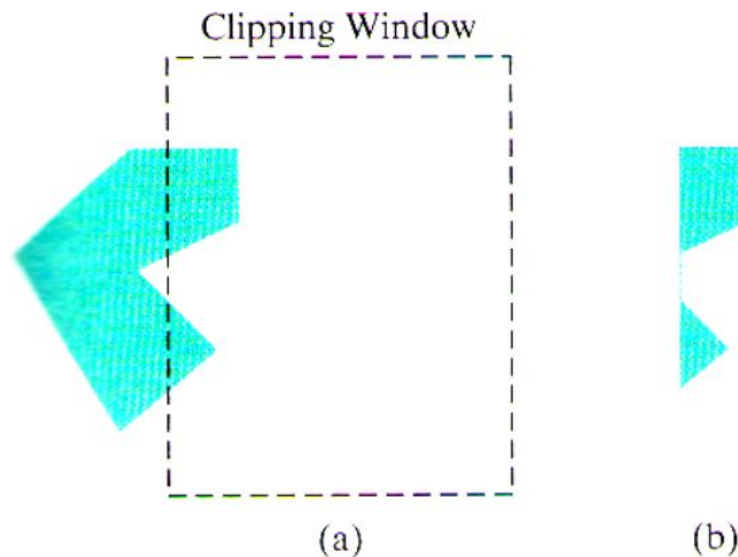


Recorte de polígonos de Sutherland-Hodgman



Recorte de polígonos de Sutherland-Hodgman

- Limitação
 - Para polígonos côncavos, problemas podem ocorrer já que esse algoritmo apenas define como saída uma única lista de vértices



- Uma solução é dividir o polígono côncavo em partes convexas, e processá-las individualmente

04

Programação OpenGL

Modo de projeção OpenGL

Definindo a janela de recorte

Definindo a Viewport

Múltiplas Viewports

Modo de projeção OpenGL

- Antes de definir a **janela de recorte** e a **viewport**, é necessário definir que a matriz em uso é a matriz de projeção

```
glMatrixMode (GL_PROJECTION) ;
```

- Não esqueça que as transformações são cumulativas, então quando necessário carregue a matriz identidade

```
glLoadIdentity() ;
```

Definindo a janela de recorte

- Janela de recorte é definida por

```
gluOrtho2D(GLfloat xmin, GLfloat xmax, GLfloat  
            ymin, GLfloat ymax);
```

- Se a janela de recorte não for especificada, as coordenadas padrão serão $xw_{\min} = yw_{\min} = -1$ e $xw_{\max} = yw_{\max} = +1$
 - O processo de recorte ocorre em um quadrado normalizado entre -1 e 1

Definindo a viewport

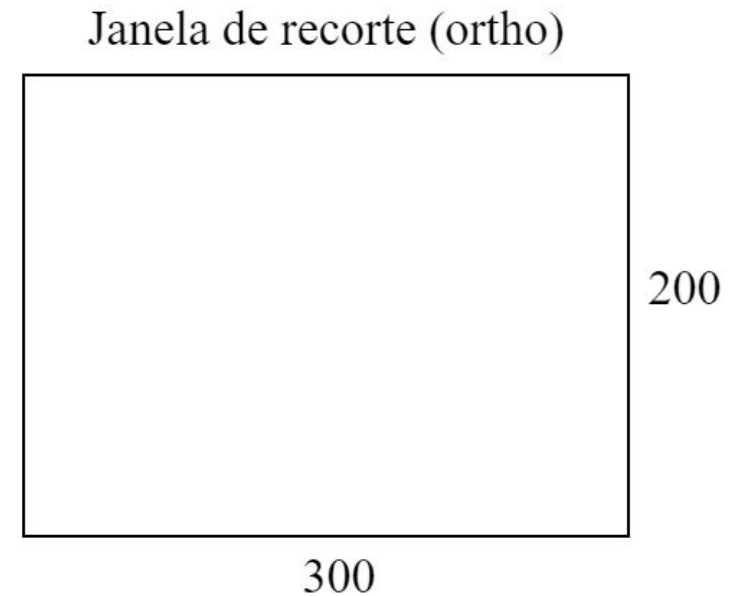
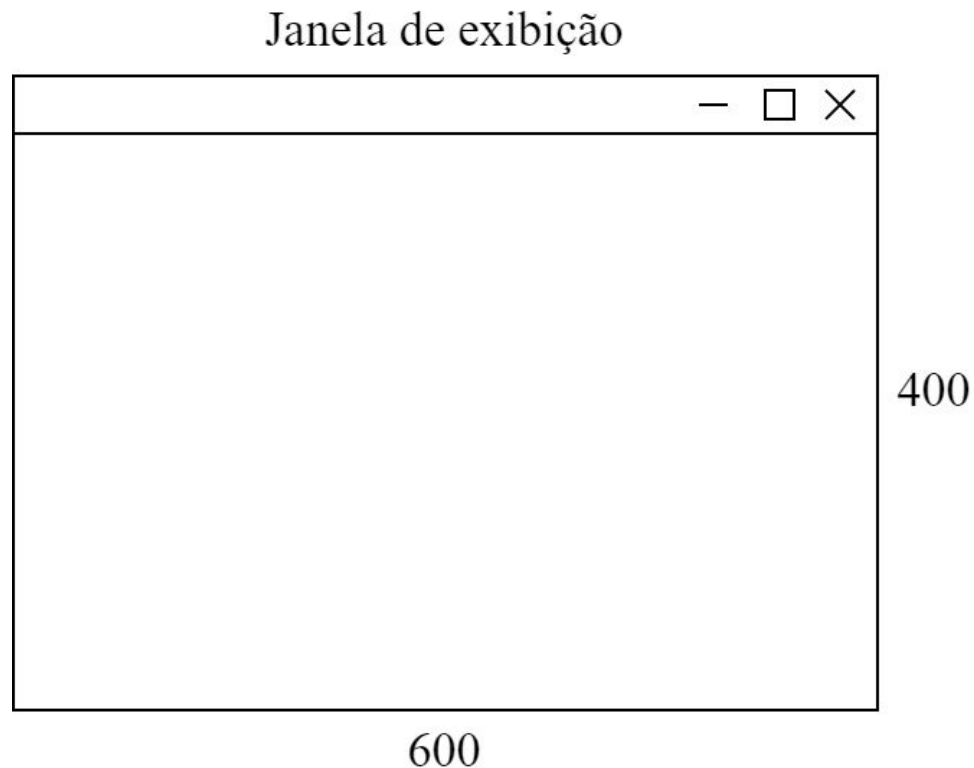
- A viewport é definida e posicionada por

```
glViewport(GLint xmin, GLint ymin, GLsizei  
           vpWidth, GLsizei vpHeight);
```

- Todos os parâmetros são dados no sistema de coordenadas da tela, relativas a janela de exibição (janela do OpenGL)
 - xmin, ymin: canto inferior esquerdo
 - vpWidth e vpHeight: largura e altura da viewport em pixels
- Se não invocarmos a função glViewport(...), a viewport irá ocupar todo o tamanho disponível na janela de exibição

Programação OpenGL

- Sem viewport declarada

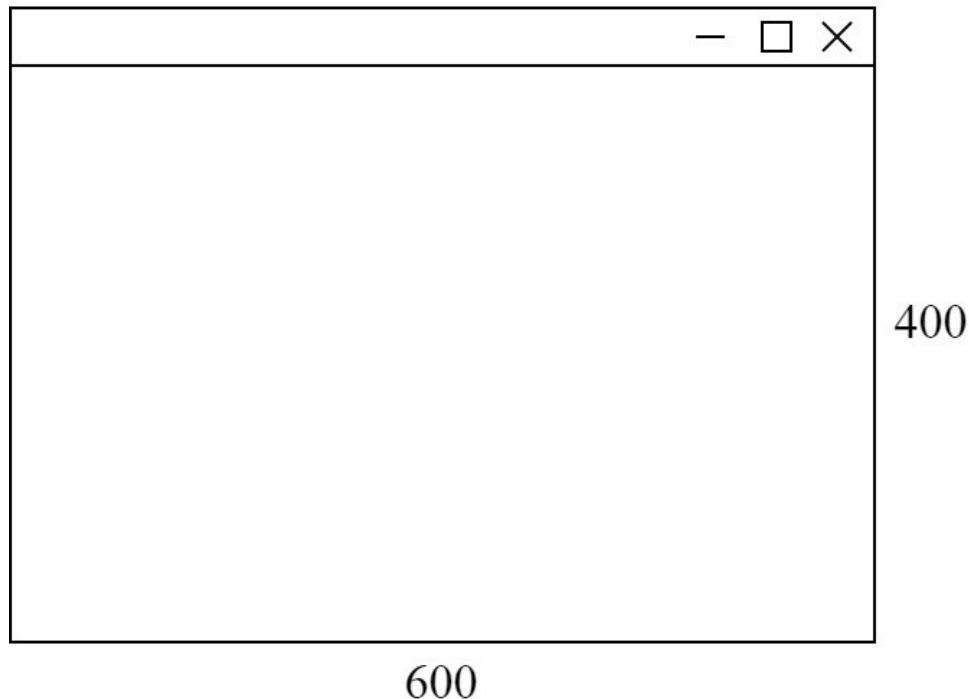


Programação OpenGL

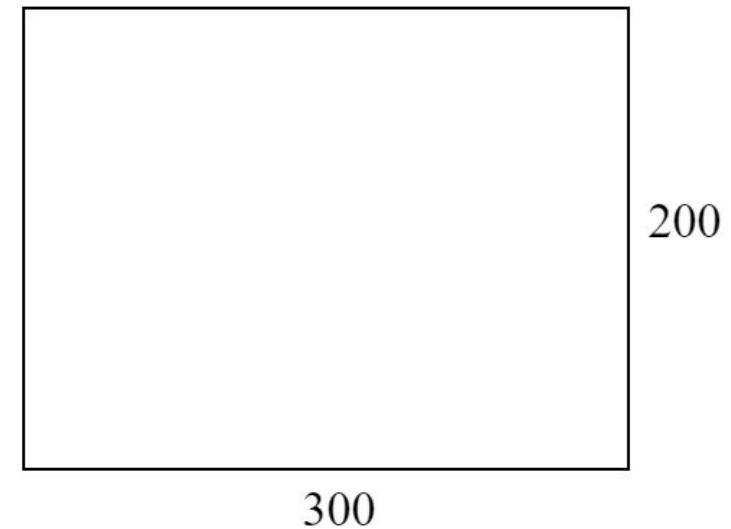
- Sem viewport declarada

Se não declararmos uma viewport, o OpenGL criará automaticamente uma ocupando todo o espaço da janela de exibição

Janela de exibição



Janela de recorte (ortho)

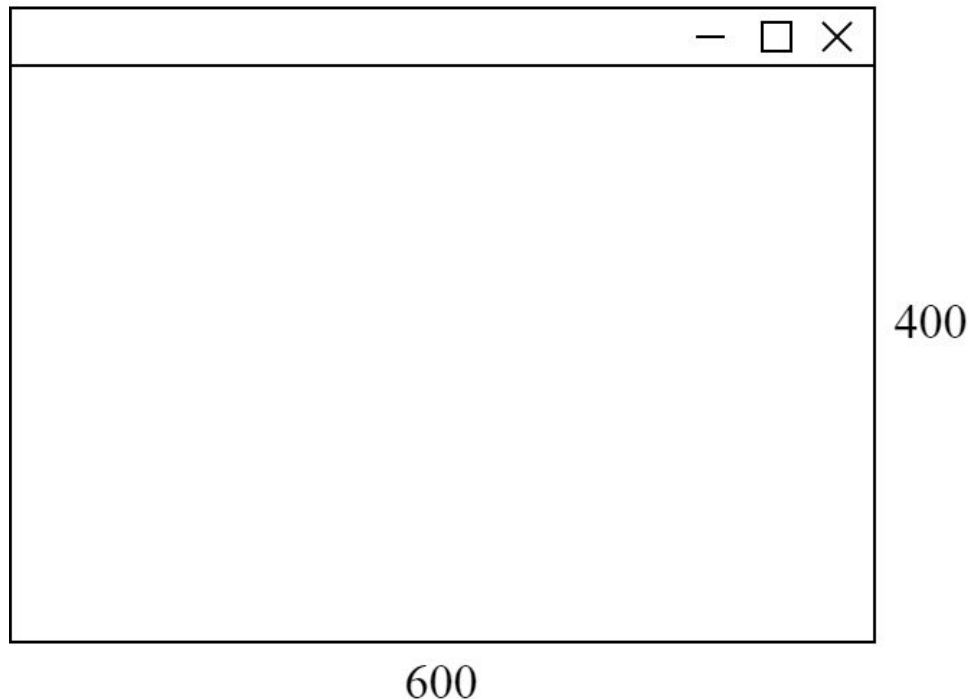


Programação OpenGL

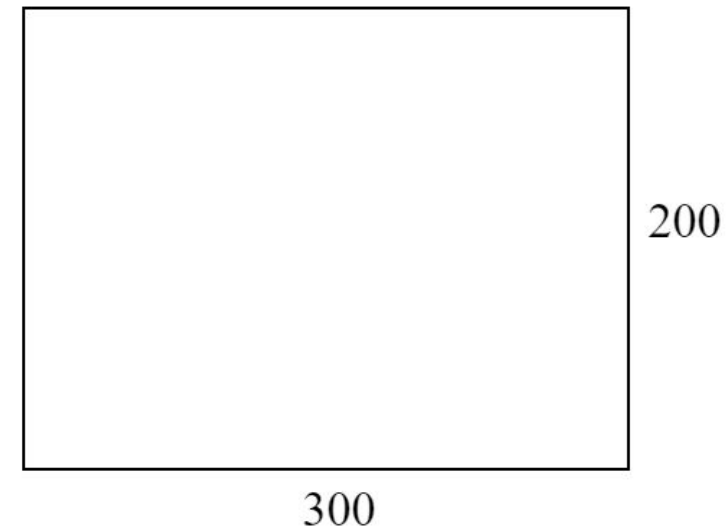
- Sem viewport declarada

Portanto, a janela de recorte e a janela de exibição devem ter a mesma razão de aspecto para não distorcer os objetos

Janela de exibição

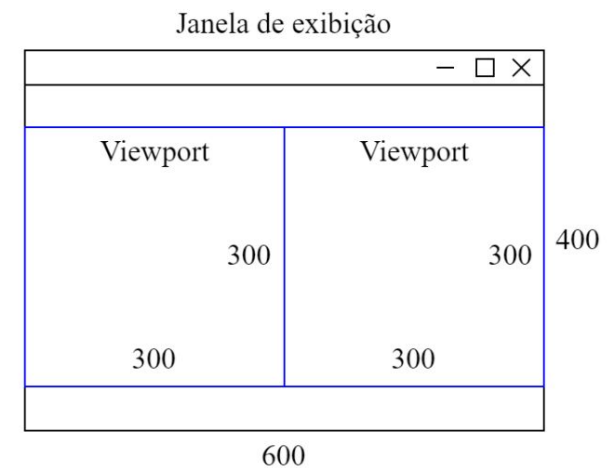
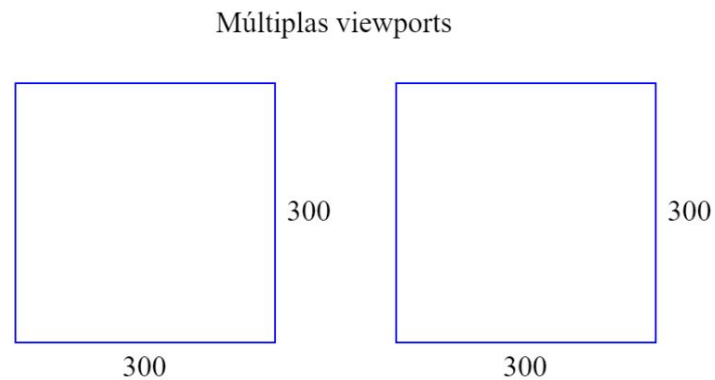
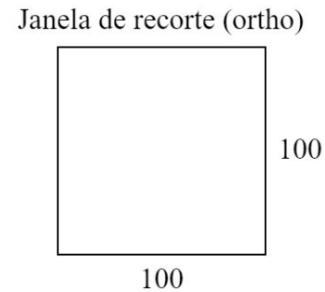
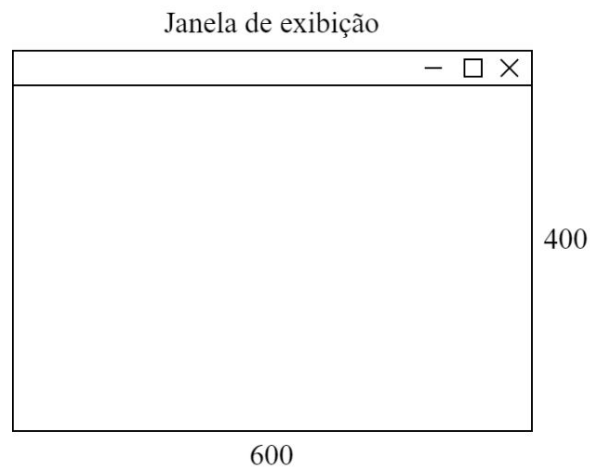


Janela de recorte (ortho)



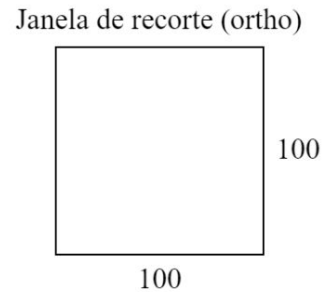
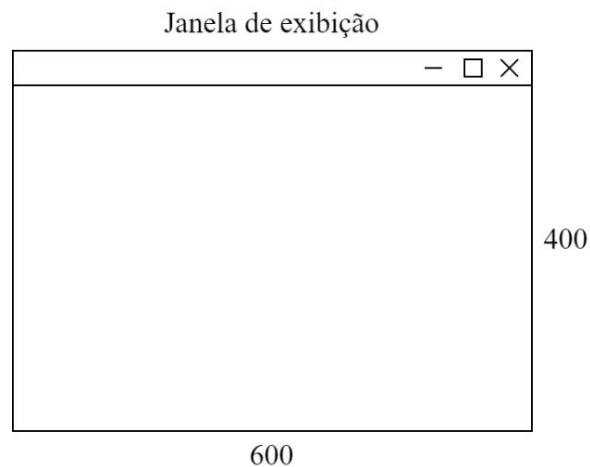
Programação OpenGL

- Com viewport declarada



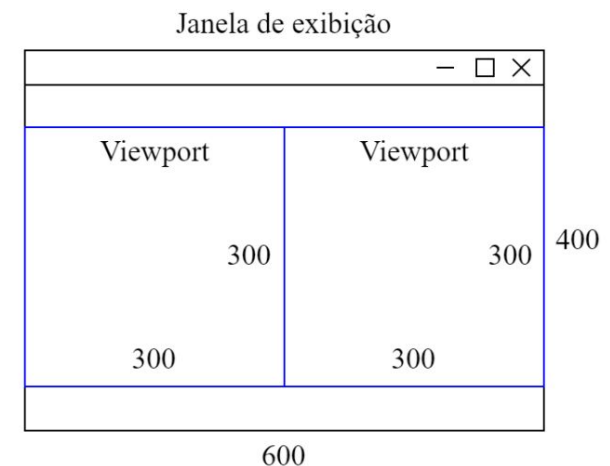
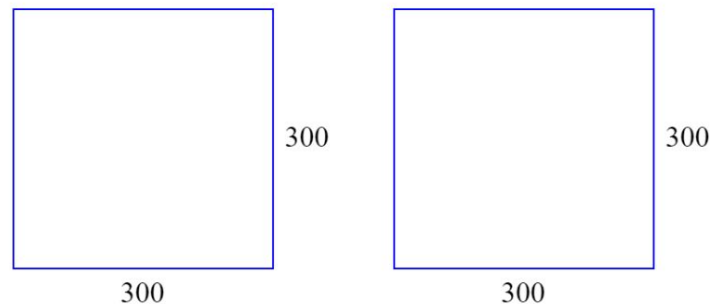
Programação OpenGL

- Com viewport declarada



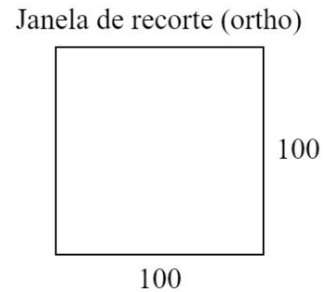
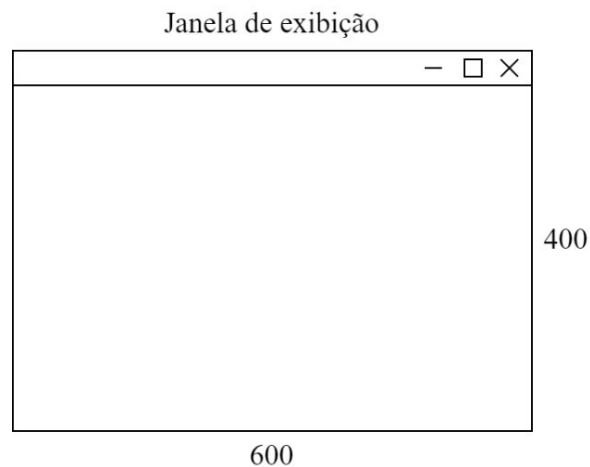
Se declararmos viewports, a janela de recorte e as viewports deverão ter a mesma proporção

Múltiplas viewports

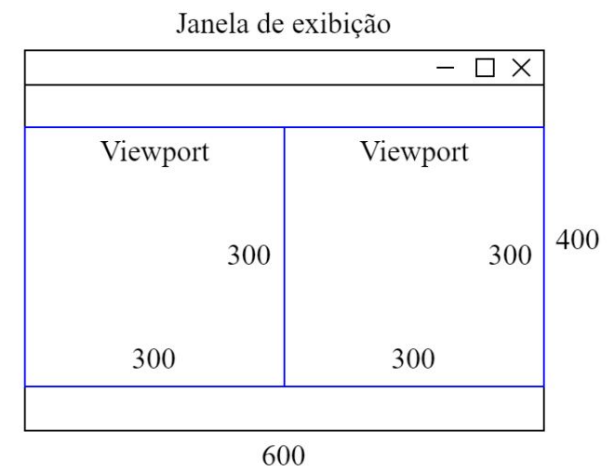
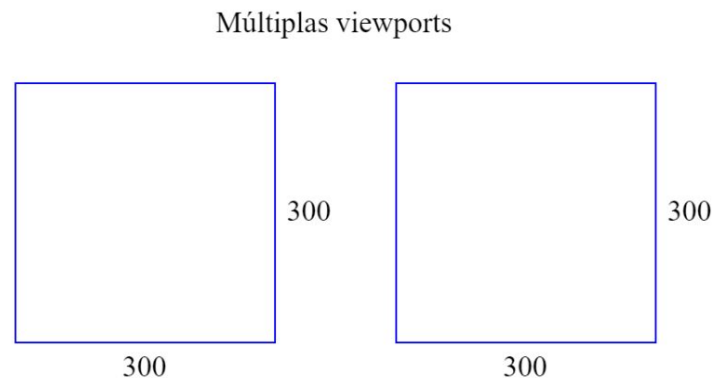


Programação OpenGL

- Com viewport declarada



E as viewports devem ser posicionadas dentro da janela de exibição



Múltiplas Viewports

Exemplo

```
int main(int argc, char** argv) {  
    glutInit(&argc, argv);  
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);  
    glutInitWindowPosition(200, 0);  
    glutInitWindowSize(500, 275);  
    glutCreateWindow("Viewing 2D - Exemplo 1");  
  
    init();  
    glutDisplayFunc(display);  
    glutMainLoop();  
    return 0;  
}  
  
//inicializa o GLUT  
//configura o modo de display  
//seta a posição inicial da janela  
//configura a largura e altura da janela de exibição  
//cria a janela de exibição  
  
//executa função de inicialização  
  
//mostre tudo e espere
```

Múltiplas Viewports

Exemplo

```
#include <windows.h>
#include <GL/glut.h>

int init() {
    glClearColor(1.0f, 1.0f, 1.0f, 0.0f); //define a cor de fundo

    glMatrixMode(GL_PROJECTION); //carrega a matriz de projeção
    glLoadIdentity(); //lê a matriz identidade
    gluOrtho2D(-100.0, 100.0, -100.0, 100.0); //define janela de recorte
}

void desenha_objeto() {
    glBegin(GL_TRIANGLES); //desenha um triângulo
        glVertex2i(50, -50);
        glVertex2i(0, 50);
        glVertex2i(-50, -50);
    glEnd();

    glBegin(GL_LINE_LOOP); //desenha um quadrado
        glVertex2i(-99, -99);
        glVertex2i(99, -99);
        glVertex2i(99, 99);
        glVertex2i(-99, 99);
    glEnd();
}

//continua...
```

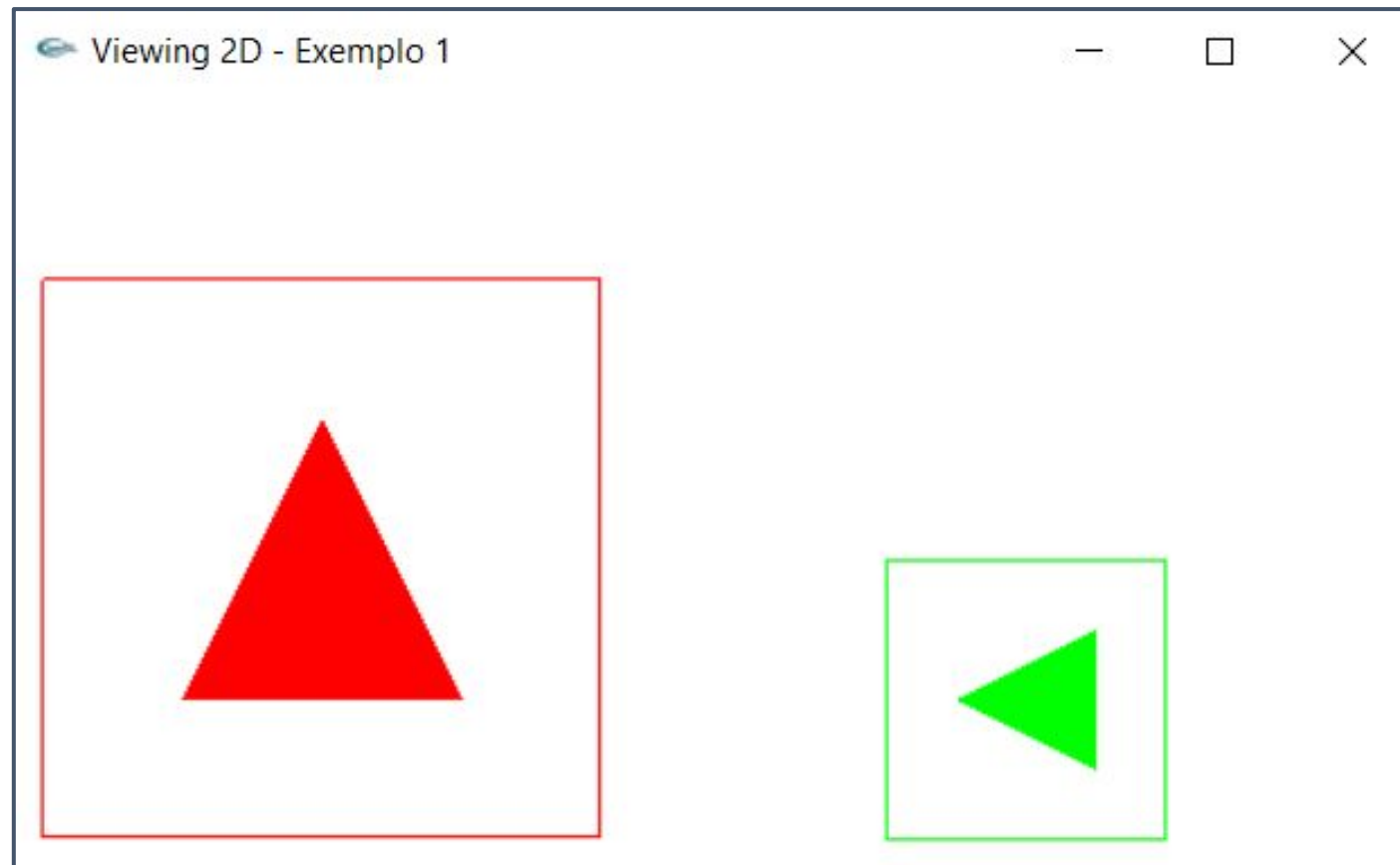
Múltiplas Viewports

Exemplo

```
void display() {  
    glClear(GL_COLOR_BUFFER_BIT); //desenha o fundo (limpa a janela)  
    glMatrixMode(GL_MODELVIEW); //matrix em uso: modelview  
    glLoadIdentity();  
  
    glViewport(10, 10, 200, 200); //define a primeira viewport  
    glColor3f(1.0f, 0.0f, 0.0f); //altera o atributo de cor  
    desenha_objeto();  
  
    glViewport(310, 10, 100, 100); //define a segunda viewport  
    glColor3f(0.0f, 1.0f, 0.0f); //altera o atributo de cor  
    glRotatef(90, 0, 0, 1);  
    desenha_objeto(); //desenha o objeto  
  
    glFlush(); //processa as rotinas OpenGL o mais rápido possível  
}  
//continua...
```


Múltiplas Viewports

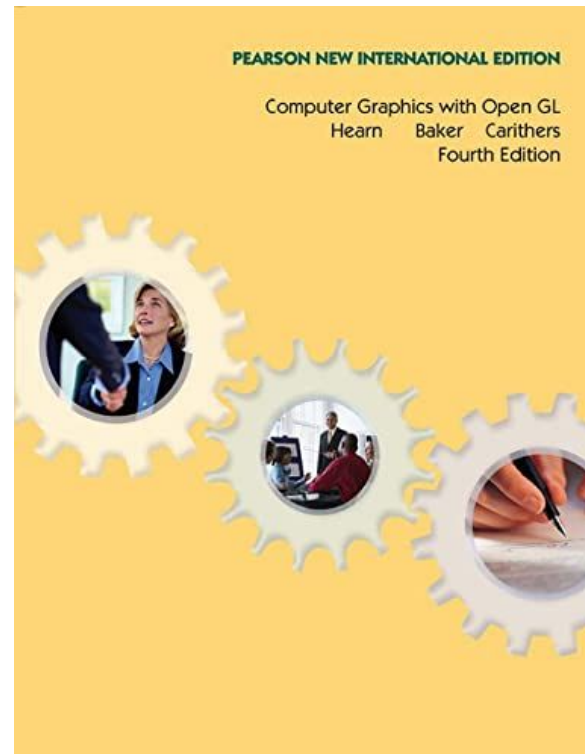
Exemplo



Referências



AZEVEDO, Eduardo; CONCI, Aura.
Computação gráfica: teoria e prática.
Elsevier, 2003.



HEARN, DONALD. **Computer graphics with OpenGL.** Pearson Education Limited, 2014.
5º Edição.



PICHETTI, R. F; et al. **Computação Gráfica e Processamento de Imagens.**
Porto Alegre: SAGAH, 2022.

Referências



- ALENCAR, Aretha. **Notas de aula da disciplina Computação Gráfica da Universidade Tecnológica Federal do Paraná.** 2021.
- PAIVA, Anselmo. **Notas de aula da disciplina Computação Gráfica da Universidade Federal do Maranhão.** 2018.
- QUINTANILHA, Darlan. **Notas de aula da disciplina Computação Gráfica da Universidade Federal do Maranhão.** 2022.
- SILVA, Giovanni. **Notas de aula da disciplina Computação Gráfica da Universidade Federal do Maranhão.** 2019.